

Lab: Scanning, Enumeration, and Sniffing

Network Discovery, Port Scanning, Service Enumeration, Intrusion Detection, and Traffic Analysis

Course: Hands-On Ethical Hacking and Network Defense — Scanning & Enumeration module companion lab

Estimated time: 2.5–3 hours **Format:** Individual; uses your isolated host-only lab (Kali attacking Metasploitable 2)

Prerequisite: Kali and Metasploitable 2 running and mutually reachable on your host-only network, with a `baseline-pre-attack` snapshot available.

1. Learning Objectives

By the end of this session you will be able to:

- Discover live hosts on a network before scanning them individually
- Run and interpret multiple Nmap scan types — TCP connect, SYN, UDP, version/OS detection — and explain when to use each
- Enumerate specific services (SMB, SNMP, NFS) to extract information beyond “open/closed”
- Configure a network intrusion detection system (Snort) to detect scanning activity as it happens
- Capture and read scan traffic at the packet level with a sniffer, and correlate it against IDS alerts
- Produce a structured, shareable report from raw scan output

2. ⚠ Scope Reminder

Same rule as every earlier session in this lab series: everything below targets your own Metasploitable 2 VM on your host-only network. This lab in particular teaches techniques — full-range scans, UDP scans, enumeration sweeps — that are loud and easy to mistake for an attack if pointed at a network you don’t control. Confirm your target’s address before every scan.

3. What You’ll Need

- Kali and Metasploitable 2 running and mutually reachable
- Root/sudo access on Kali (several scan types require it)
- A restored `baseline-pre-attack` snapshot of Metasploitable 2

4. Procedure

Part A — Discover What’s Actually on the Network

1. Before scanning a specific host, find out what’s alive. Run a ping sweep across your host-only subnet:

```
nmap -sn 192.168.56.0/24
```

2. Cross-check with an ARP-based scanner, which works even against hosts that block ICMP:

```
sudo arp-scan --localnet
```

3. Record every host you find, with its IP and (if shown) MAC vendor. Confirm Metasploitable’s address matches what you recorded in earlier sessions.

Part B — Port Scanning, Scan Type by Scan Type

Run each of the following against Metasploitable 2 and record how long each takes and what it reports differently from the others.

1. **TCP connect scan** (completes the full three-way handshake — reliable, but noisy and logged everywhere):

```
nmap -sT 192.168.56.102
```

2. **SYN (“stealth”) scan** (sends SYN, reads the response, never completes the handshake — needs root):

```
sudo nmap -sS 192.168.56.102
```

3. **UDP scan** (slower and less reliable — closed UDP ports often just don’t respond):

```
sudo nmap -sU --top-ports 20 192.168.56.102
```

4. **Version and OS detection**, combined into one pass:

```
sudo nmap -sV -O 192.168.56.102
```

5. **Full port range**, so nothing outside the common 1,000 is missed:

```
nmap -p- 192.168.56.102
```

Reflection (2-3 sentences): Which scan type found the most open ports? Which took longest? Why would a real engagement ever choose the slower, noisier option over the faster one?

Part C — Enumeration: Going Past “Open”

An open port only tells you a service is listening. Enumeration extracts what that service will actually hand over.

1. **SMB/NetBIOS** — list shares without authenticating:

```
smbclient -L //192.168.56.102/ -N  
enum4linux -a 192.168.56.102
```

Record every share name and any share that allows anonymous access.

2. **SNMP** — Metasploitable ships SNMP with a commonly-used default community string. Try it:

```
snmpwalk -c public -v1 192.168.56.102
```

Record what system information comes back with no authentication beyond guessing the community string.

3. **NFS** — list exported filesystems:

```
showmount -e 192.168.56.102
```

4. **Banner grabbing** — connect directly to a service and read what it volunteers:

```
nc -nv 192.168.56.102 21
```

Record the banner. Compare it to what Nmap's `-sV` reported in Part B — do they match?

Part D — Set Up an IDS to Watch Your Own Scans

1. Install Snort if it isn't already present:

```
sudo apt install snort
```

During setup, set your home network to your host-only subnet (`192.168.56.0/24`).

2. Run Snort in simple IDS mode, watching your host-only interface and printing alerts to the console:

```
sudo snort -i eth0 -A console -q -c /etc/snort/snort.conf
```

(Substitute your actual host-only interface name if it isn't `eth0`.)

3. In a second terminal, re-run your Part B SYN scan against Metasploitable while Snort is running.
4. Watch the first terminal. Record whether Snort raised any alerts, and if so, what they said.
5. If nothing fired, that's a real and common finding, not a failure — Snort's default ruleset doesn't alert on everything out of the box. Add one simple custom rule to `/etc/snort/rules/local.rules`:

```
alert tcp any any -> 192.168.56.102 any (msg:"Possible Nmap SYN scan detected"; flags:S; threshold: type  
threshold, track by_src, count 20, seconds 5; sid:1000001;)
```

Restart Snort and re-run the scan. Record the alert this time.

Part E — Sniff the Same Traffic

1. Start a capture in Wireshark (or `tcpdump -i eth0 -w scan_capture.pcap`) on your host-only interface.
2. Re-run the Part B SYN scan one more time while the capture is running.
3. Stop the capture. In Wireshark, filter on:

```
tcp.flags.syn==1 and tcp.flags.ack==0
```

4. Record what you see: a burst of SYN packets to many destination ports in a very short window, each part of the same scan. This is the signature Snort's rule in Part D was written to catch.
5. **Reflection (2-3 sentences):** You just observed the same event three ways — as scan output (Part B), as an IDS alert (Part D), and as raw packets (this part). Which view would a defender actually see first in a real environment, and which one gives them the most certainty that it wasn't a false alarm?

Part F — Generate a Report

1. Re-run one full scan with all output formats saved at once:

```
sudo nmap -sV -O -p- -oA metasploitable_scan 192.168.56.102
```

This produces three files: `.nmap` (human-readable), `.xml` (structured), and `.gnmap` (greppable).

2. Convert the XML report into a formatted HTML report using Nmap's bundled stylesheet:

```
xsltproc metasploitable_scan.xml -o metasploitable_scan.html
```

3. Open the HTML report in a browser and confirm it's readable — this is the format you'd actually hand to a client or a manager, not raw terminal output.
4. Write a one-page cover summary to sit in front of that report, covering:
 - Scope: what you scanned and when
 - Headline finding: the single most significant thing from Parts B-D (e.g., anonymous SMB shares, the SNMP community string, or whether your scan was detected)
 - Three prioritized recommendations, ranked by what would reduce risk fastest

5. What to Submit

1. Part A host discovery results
2. Part B scan outputs for all five scan types, plus your reflection answer
3. Part C enumeration findings (SMB shares, SNMP output, NFS exports, one banner grab)
4. Part D — whether the default ruleset caught your scan, and the alert output after adding your custom rule
5. Part E — your Wireshark SYN-burst filter screenshot, plus your reflection answer
6. Part F — your generated HTML report and one-page cover summary

6. Troubleshooting

Symptom	Likely cause / fix
<code>sudo nmap -sS</code> behaves identically to <code>-sT</code>	You're not actually running as root — confirm <code>sudo</code> took effect
<code>enum4linux</code> or <code>snmpwalk</code> not found	Install with <code>sudo apt install enum4linux snmp</code>
Snort won't start / config errors	Snort's default config often needs its network variables (<code>HOME_NET</code>) edited in <code>/etc/snort/snort.conf</code> before first run
No Snort alerts even with the custom rule	Confirm Snort is watching the correct interface (<code>ip addr</code> to check), and that the scan is actually reaching Metasploitable during the capture window

This lab is a standalone Scanning, Enumeration & Sniffing session — like the Footprinting lab, it slots in wherever your course covers this module. Unlike that lab, everything here runs inside your isolated host-only network — no internet access needed.